KODEXIA · Software & ERP Solutions

Documento de Diseño de Software (SDD)

AI Sales Assistant API & Arquitectura de Contenedores

Autor: Kodexia, Software & ERP Solutions

Entorno: Docker Core & FastAPI Backend & Ollama Local Service

Fecha: Octubre 2026

Versión: 1.1.0 (Estabilizada)

Estado: Aprobado para Producción

Clasificación: CONFIDENCIAL - Uso Interno

Este documento complementa, sin repetirlos, la arquitectura (docs/specs/08-architecture.md), el diseño de la API (docs/specs/07-api-design.md) y los ADR (docs/adr/README.md).

1. Introducción y Propósito

Este Documento de Diseño de Software (SDD) describe la arquitectura técnica, diseño de datos e integración de la API del AI Sales Assistant desarrollado por **Kodexia**. La aplicación se ejecuta en un contenedor Docker y expone servicios web multilingües con soporte de Inteligencia Artificial local a través de Ollama, que corre en el equipo anfitrión (host).

1.1 Alcance del Sistema

La API gestiona la autenticación de usuarios (incluidos los flujos de invitados), la persistencia de datos relacionales, la interfaz en cuatro idiomas (i18n), la mitigación de fallos de red en las pruebas y la generación de textos comerciales en Español, Inglés, Portugués y Francés con modelos locales compactos (`llama3.2:3b`), sin depender de proveedores cloud externos (OpenAI queda como proveedor alternativo opcional).

La salida del modelo no es determinista: lo determinista es la **validación numérica** que el backend aplica después (sección 5.2).

---

2. Arquitectura del Sistema y Contenedores

Solo la aplicación está contenedorizada. Ollama **no** corre en un contenedor: se ejecuta en el host y usa directamente su CPU/GPU. El contenedor lo alcanza a través de `host.docker.internal` .

```
Navegador ──HTTP :8000──► contenedor "mi-api-assistant" (python:3.12-slim)
                          uvicorn app.main:app --reload
                          carpeta del proyecto montada en /app (código, .env, SQLite)
                                │
                                ├──► SQLite ./ai_sales_assistant.db (el mismo archivo que fuera de Docker)
                                │
                                └──HTTP──► host.docker.internal:11434 ──► Ollama en el host
                                           (o la API de OpenAI si AI_PROVIDER=openai)
```

Componente	Tecnología / Imagen	Propósito y Enlace de Red
Backend Web API	FastAPI (`python:3.12-slim` / Uvicorn)	Expone los endpoints REST y sirve la interfaz visual en el puerto 8000.
Servicio de IA	Ollama en el host (no contenedorizado)	Sirve el modelo `llama3.2:3b` . El contenedor lo alcanza en `http://host.docker.internal:11434` .

Componente	Tecnología / Imagen	Propósito y Enlace de Red
Base de Datos	SQLite (migraciones Alembic hasta la 0006)	Archivo en la carpeta del proyecto, montada en el contenedor: los datos sobreviven a la reconstrucción del contenedor.

Las interferencias de software de seguridad del host (firewalls, antivirus) **no** las elimina la red de Docker. Se mitigan en las pruebas end-to-end con la política de reintento basada en evidencias de la sección 6.1.

2.1 Red de Docker

Dentro de un contenedor, `localhost` es el propio contenedor, no el host. Por eso `docker-compose.yml` define:

```
environment:
  OLLAMA_BASE_URL: http://host.docker.internal:11434   # el equipo anfitrión
extra_hosts:
  - "host.docker.internal:host-gateway"                # también funciona en Linux
```

- **Precedencia:** las variables de entorno prevalecen sobre `.env`. El valor de `.env` (`http://localhost:11434`) sigue siendo correcto fuera de Docker y no hay que cambiarlo.
- `host.docker.internal` lo define Docker Desktop (Windows/macOS) y `host-gateway` lo añade en Linux. `docker.internal` (sin `host.`) no existe y no resuelve.
- **Arranque con `docker compose up`**, no con `docker run`: un contenedor creado con `docker run` no recibe el bloque `environment`, usa el valor de `.env` y se llama a sí mismo.
- Un cambio en `environment` exige **recrear** el contenedor (`docker compose up -d --force-recreate app`); `docker restart` conserva las variables antiguas.

Diagnóstico desde el host:

```
docker exec mi-api-assistant sh -c 'env | grep OLLAMA'          # debe mostrar host.docker.internal
docker exec mi-api-assistant python -c "import urllib.request as u; \
  print(u.urlopen('http://host.docker.internal:11434/api/tags', timeout=5).status)"   # 200
```

---

3. Diseño de la API y Canales de Comunicación

La API expone documentación viva a través de OpenAPI (Swagger UI) en `/docs`. La interfaz gráfica usa estilos minimalistas y estados responsive basados en los estándares de **Kodexia | Software & ERP Solutions**. Las rutas `/api/v1` responden JSON y están documentadas en OpenAPI; las rutas web devuelven HTML y quedan fuera de OpenAPI.

3.1 Endpoints Core

- `GET /`: Estado del servicio backend en formato JSON plano.

- `GET /docs`: Documentación interactiva de endpoints y esquemas Pydantic.
- `GET/POST /login`: Inicio de sesión (también funciona sin JavaScript); las credenciales de demostración abren una sesión de invitado.
- `POST /guest` y `POST /api/v1/auth/guest`: Acceso rápido como Invitado (cuenta temporal).
- `GET/POST /contacto`: Formulario web con validación obligatoria anti-inyecciones HTML y cumplimiento RGPD.
- `GET /app`: Panel principal del asistente donde se parametriza la generación (redirige a `/login` sin sesión).
- `POST /api/v1/generate`: Generación comercial con IA (admite el `customer_email` opcional, sección 4.3).
- Acciones instantáneas del resultado: Correo (`mailto:`) y WhatsApp (`wa.me`), en el navegador (sección 3.4).

3.2 API completa

Método	Ruta	Sesión	Propósito
GET	`/`	—	Información de la API
GET	`/health`	—	Estado (también lo usa el servidor de las pruebas e2e)
POST	`/api/v1/auth/register`	—	Registro (abre sesión y adopta las generaciones del invitado)
POST	`/api/v1/auth/login`	—	Inicio de sesión con usuario o email
POST	`/api/v1/auth/guest`	—	Entrar como invitado
POST	`/api/v1/auth/logout`	sí	Cerrar sesión (un invitado se borra con sus datos)
GET	`/api/v1/auth/me`	sí	Usuario actual
POST	`/api/v1/auth/change-password`	registrado	Cambiar contraseña (cierra las demás sesiones)
POST	`/api/v1/auth/forgot-password`	—	Pedir enlace de restablecimiento (misma respuesta exista o no la cuenta)
POST	`/api/v1/auth/reset-password`	—	Restablecer la contraseña con el token
POST	`/api/v1/generate`	sí	Generar contenido (con `customer_email` opcional)
GET	`/api/v1/generations`	sí	Listar las generaciones del usuario (paginado, filtro por tipo)
GET	`/api/v1/generations/{id}`	sí	Ver una (la de otro usuario responde 404)
PUT	`/api/v1/generations/{id}`	sí	Regenerar con datos nuevos
DELETE	`/api/v1/generations/{id}`	sí	Eliminar
GET	`/api/v1/pricing/discount`	sí	Desglose del descuento calculado por el backend

3.3 Rutas web

Método	Ruta	Propósito
GET	`/login`, `/register`, `/forgot-password`, `/reset-password`	Pantallas de acceso
POST	`/login`, `/guest`	Formularios clásicos: funcionan sin JavaScript
GET	`/app`, `/account`	Generador y cuenta (redirigen a `/login` sin sesión)
GET	`/language/{code}?next=/ruta`	Cambio de idioma de la interfaz (sección 5.3)
GET / POST	`/contacto`	Datos de soporte y formulario de contacto
GET	`/legal`, `/privacidad`	Aviso legal y política de privacidad

Las peticiones que cambian estado pasan un control de origen (CSRF) y todas las respuestas llevan cabeceras de seguridad (ver arquitectura (docs/specs/08-architecture.md)).

3.4 Acciones instantáneas del resultado (Correo y WhatsApp)

En la cabecera de la tarjeta «Resultado», junto a «Copiar», hay dos botones con icono lineal (Correo y WhatsApp), del mismo tamaño y estilo (contorno en el teal de la marca). No llaman a la API: son enlaces cuyo `href` se reconstruye en el navegador con cada resultado.

- **Estado:** desactivados (sin `href`, `aria-disabled="true"`, fuera del orden de tabulación) hasta que hay un resultado; se activan al mostrarlo y vuelven a desactivarse mientras se genera otro.
- **Correo:** `mailto:<customer_email>?subject=Propuesta%20Comercial%20-%20Kodexia&body=<texto>`. El destinatario es el `customer_email` de la generación; si está vacío, el gestor de correo se abre sin destinatario. El asunto es fijo («Propuesta Comercial - Kodexia») y el cuerpo es el texto plano del resultado.
- **WhatsApp:** `https://wa.me/?text=<texto>` en una pestaña nueva. Sin número de teléfono (el formulario no lo pide): WhatsApp abre el selector de chats.
- **Codificación:** el texto pasa por `encodeURIComponent`, de modo que `&`, `?`, `#` o saltos de línea no pueden romper el enlace ni inyectar parámetros (por ejemplo un segundo `subject`).
- **Implementación:** `shareLinks(texto, email)` en `app/static/js/generation-form.js` (función pura, probada con Node); `app.js` la aplica al renderizar cada resultado.

---

4. Modelo de Datos, Validación y Cumplimiento Normativo

Para garantizar la seguridad de la información y cumplir con las regulaciones de la Unión Europea (RGPD/LOPDGDD), el sistema aplica el modelo de información por capas de la AEPD en el formulario de contacto.

4.1 Flujo de Consentimiento y Sanitización

1. **Casilla de Verificación Obligatoria:** El formulario expone un checkbox desmarcado por defecto para la aceptación explícita de la política de privacidad. Se valida también en el servidor (`ContactMessage.privacy_accepted`, error `privacy_not_accepted`), no solo en el navegador.
2. **Información en Primera Capa** (`/contacto`): junto al formulario se muestran el Responsable, la Finalidad (gestión de consultas), la Legitimación por consentimiento, los Destinatarios, los Derechos y el enlace a la información adicional. Los datos que aún no existen (responsable, destinatarios) muestran «Pendiente de completar»: no se inventan.
3. **Segunda Capa** (`/privacidad`): la política completa, una sección por apartado (responsable, datos, finalidad, base jurídica, conservación, destinatarios, derechos y contacto de privacidad).
4. **Sanitización Estricta:** El backend rechaza las etiquetas HTML y los caracteres `<` y `>` para prevenir inyecciones de código (XSS); asunto y nombre deben ir en una sola línea y sin caracteres de control.

4.2 Minimización de datos

- El `customer_email` de una generación **nunca se envía al proveedor de IA**: el prompt solo incluye los campos que lista `app/prompts/builder.py`, y un test lo garantiza (sección 4.3).
- `CONTACT_FORM_MODE=demo` (único modo por ahora): el mensaje se valida y se **descarta**. No se envía, no se guarda y no se registra en los logs, y la página lo dice en lugar de fingir el envío (`app/services/contact.py`).
- Las cuentas de invitado y sus generaciones se borran al salir o al caducar.
- Ningún secreto llega a una página: el token de restablecimiento se quita de la barra de direcciones tras leerlo, y el selector de idioma enlaza a la **ruta sin la query**, de modo que `/reset-password?token=…` nunca se copia en la página (cubierto por un test).

Previsto (modo «email»): envío con el `EmailSender` existente (SMTP) a un buzón de soporte, con límite de envíos y honeypot.

4.3 Campo `customer_email` (migración 0006)

- **Para qué:** rellena el destinatario del botón de Correo (sección 3.4). Es opcional.
- **Formulario:** «Email del cliente (opcional)», tras «Producto o servicio»; `type="email"`, máximo 254 caracteres.
- **API:** `GenerationCreate.customer_email` valida un email real (`EmailStr`, ≤ 254); vacío o solo espacios equivale a «no indicado» (`null`). Un email inválido responde 422 con el detalle en el campo, y la página muestra «El email no es válido.». `GenerationRead` lo devuelve, «Editar» lo restaura y `PUT` lo actualiza.
- **Base de datos:** columna `generations.customer_email` (`VARCHAR(254)`, nullable), añadida por la migración Alembic **0006** (`alembic/versions/0006_add_generation_customer_email.py`). Las generaciones existentes conservan todos sus datos con el campo vacío; el downgrade elimina solo esa columna.
- **Privacidad:** se guarda con la generación de su propietario (y se borra con ella o con la cuenta de invitado) y no forma parte del prompt.

---

5. Ingeniería de Prompts e Internacionalización (i18n)

El motor de i18n se integra en las plantillas mediante diccionarios locales (`es.py` , `en.py` , `pt.py` , `fr.py`). Las pantallas de acceso ocupan al menos el alto de la ventana (`min-height: 100vh`) en todos los idiomas, con el scroll vertical permitido cuando el contenido no cabe.

5.1 Lógica Imperativa en el Prompt del Sistema

Para solventar las limitaciones del modelo compacto `llama3.2:3b` al mezclar contextos en español con solicitudes de salida en otro idioma, `app/prompts/base.py` (`language_rule`) genera una regla imperativa que **abre el prompt del sistema y cierra el prompt del usuario** (y el del reintento). Nombra el idioma pedido y prohíbe **los otros tres**. Por ejemplo, para inglés:

> *CRITICAL: You MUST write the final response entirely in the requested language (English). Do not write in Spanish, Portuguese, French, even if the input context data is in another language: translate it into English. The subject line, greeting, body and closing must all be in English. This includes the mandatory commercial terms: write them in English too, keeping every figure exactly as written.*

Resultados medidos con `llama3.2:3b` y datos de entrada en español: 7/8 salidas en el idioma pedido para inglés y 4/4 para portugués y francés. Los deslices ocasionales son un límite del modelo; un modelo mayor u OpenAI los eliminan.

5.2 Validación Numérica Determinista

Para asegurar la veracidad comercial sin romper la traducción, cada línea de `<mandatory_commercial_terms>` debe aparecer en el idioma de salida con **todas sus cifras exactamente como se escribieron**. La verificación del backend (`find_missing`) no compara texto: compara solo las cifras (los números por su valor, las fechas literalmente), así que una línea traducida pasa y una cifra cambiada falla. Si falta algún término se reintenta una vez (`COMMERCIAL_TERMS_MAX_RETRIES`); en modo estricto el texto se rechaza en lugar de guardarse con aviso.

5.3 Idiomas de la interfaz

Español (por defecto), Inglés, Portugués (pt-PT) y Francés. El idioma de la interfaz es independiente del idioma del contenido generado, que es un campo del formulario.

```
Navegador ── GET /language/en?next=/login ──► cookie ui_lang=en (1 año, HttpOnly, SameSite=Lax)
                                              303 → /login
Petición ── resolve_language(cookie) ──► translator(t) ──► plantillas: {{ t("clave") }}
                                    └──► js_messages("en") ──► <script id="i18n-messages"> ──► static/js/i18n.js
```

- **Catálogos:** `app/i18n/{es,en,pt,fr}.py` , un diccionario `MESSAGES` con las mismas claves. `es.py` es la fuente; una clave que falta cae al español y una clave desconocida se muestra tal cual, para que los tests la detecten.

- **Servidor:** `render()` entrega a cada plantilla `t`, `ui_language` (`<html lang>`), los datos del selector y `js_messages`. Las opciones de los selectores son claves (`option.content_type.*`, `option.language.*`, `option.tone.*`): los valores que se envían a la API no cambian.
- **JavaScript:** `static/js/i18n.js` expone `t(clave, textoEspañol, params)` y `locale` (`es-ES`, `en-GB`, `pt-PT`, `fr-FR`) para números y fechas. El texto español de respaldo permite usar y probar los módulos con Node sin página.
- **Selector:** en las cabeceras de acceso, públicas y de la aplicación; cada idioma enlaza a `/language/<código>`.
- **Añadir un idioma:** crear `app/i18n/<código>.py` con todas las claves, registrarlo en `SUPPORTED_LANGUAGES` y `CATALOGS` y añadir su locale a `i18n.js`.

Garantías (`tests/test_i18n.py`):

- todos los catálogos tienen las mismas claves y los mismos `{marcadores}`;
- cada texto de respaldo del JavaScript es igual al del catálogo español;
- ninguna página renderizada en cada idioma contiene texto del catálogo español;
- los textos del pie de página están fijados por idioma.

---

6. Estrategia de Pruebas y Tolerancia a Fallos

El sistema incorpora una suite automatizada de 821 pruebas unitarias/API, 98 pruebas de JavaScript y 23 escenarios End-to-End (e2e) ejecutados sobre navegadores Chrome controlados de manera aislada, además de 6 pruebas opcionales contra el modelo real.

Cobertura de las acciones instantáneas y del `customer_email`:

- **API:** se guarda y se devuelve, es opcional, un email inválido o demasiado largo se rechaza, `PUT` lo actualiza y nunca aparece en el prompt enviado a la IA.
- **Migración 0006:** las generaciones existentes quedan con el campo vacío y el downgrade conserva las generaciones.
- **JavaScript:** `shareLinks` construye el `mailto:` (destinatario, asunto «Propuesta Comercial - Kodexia», cuerpo), funciona sin destinatario, el texto no puede romper el enlace y el enlace de `wa.me` lleva el texto.
- **e2e:** ambos botones aparecen justo después de «Copiar», en la misma fila y con la misma altura, desactivados antes del resultado y con los enlaces `mailto:` y `wa.me` correctos después; «Editar» restaura el email.

Suite	Comando	Notas
Unitarias y API	`pytest --ignore=tests/e2e --ignore=tests/llm`	También en el contenedor
JavaScript	`node --test tests/js/*.test.mjs`	Módulos puros, sin navegador
e2e en navegador	`pytest tests/e2e`	Necesita Node.js y Chrome en el host
Modelo real	`RUN_LLM_TESTS=1 pytest tests/llm -s`	Llama al proveedor configurado

Los e2e arrancan un servidor uvicorn real en un **puerto libre** (`free_port()` , en 127.0.0.1), con una base SQLite desechable, un Ollama falso y un entorno limpio (sin `.env` ni variables de OpenAI o SMTP). Un arnés en Node (`tests/e2e/browser_flows.mjs`) controla Chrome sin interfaz mediante el protocolo DevTools.

6.1 Política de Reintento Automático de Servidor

En Windows, la inspección de un antivirus o firewall a veces retiene una conexión local y la página nunca llega. Reintentar a ciegas ocultaría errores reales, así que el módulo `tests/e2e/server_log.py` solo permite un reintento bajo estas condiciones exclusivas: * La prueba falla y el arnés informa de las peticiones que Chrome dejó esperando (una página que no cargó en 15 s o peticiones pendientes). * El registro del propio servidor demuestra que **ninguna** de esas peticiones llegó (caída de red o bloqueo del puerto). * *Nota:* Si el fallo se debe a una aserción, a un error de JavaScript o a cualquier petición que sí llegó al servidor, el test falla inmediatamente sin reintentos.

```
falla el escenario
   └─ ¿qué peticiones dejó esperando Chrome?
        ├─ ninguna informada ..................................... fallo real → falla
        ├─ alguna aparece en el log del servidor ................. el servidor está implicado → falla
        └─ ninguna aparece en el log del servidor ................ nunca llegaron → se reintenta UNA vez, con aviso
```

- El log del servidor no usa búfer (`PYTHONUNBUFFERED=1`) y se escribe en un archivo, de modo que la decisión ve todas las peticiones ya servidas; solo se lee lo escrito durante el intento fallido.
- Dentro de una ejecución, una página que no llega en 15 s se pide de nuevo **una vez** (es un GET y se puede repetir). Los clics y los envíos de formularios nunca se repiten.
- Un segundo fallo siempre hace fallar el test, y cada reintento aparece como aviso en pytest.
- El arnés solo cierra procesos de Chrome cuyo `--user-data-dir` es exactamente su propio perfil temporal (`<temp>/e2e-chrome-XXXXXX`): el Chrome del usuario nunca coincide. Cada cierre se registra y el test lo audita.

Las reglas de decisión tienen sus propios tests en `tests/test_e2e_retry_policy.py` .

---

7. Estilos e Identidad Corporativa

Toda la identidad de componentes dinámicos de la interfaz se centraliza en `app/static/css/brand.css` , con los colores definidos como variables `--brand-*` .

- **Paleta de Colores:** Los estados activos, enfoques y selecciones (`:hover` , `:focus` , `option:checked`) de los menús desplegables se unifican bajo la tonalidad teal oscura de la marca, `#0b6f6b` (`--brand-teal-strong`), con texto en blanco (contraste 6,0:1, WCAG AA). En el pie de página minimalista, los enlaces y sus iconos están en gris suave en reposo y al pasar el cursor o con el foco solo el texto y el icono se tiñen de `#0b6f6b` , sin fondos ni cajas.

- **Estructura de pantalla completa:** Las pantallas de acceso usan un layout flexible con `min-height: 100vh`: ocupan al menos toda la ventana en escritorio y permiten el scroll vertical cuando el contenido no cabe (por ejemplo, en móvil).
- **Pie de página:** una sola fila con el copyright, los enlaces legales y los enlaces Email / WhatsApp / GitHub, configurados con `SUPPORT_EMAIL`, `SUPPORT_PHONE` (abierto con `wa.me`) y `GITHUB_URL`; un valor vacío oculta su enlace.

---

8. Limitaciones conocidas

- El botón de WhatsApp no lleva número (el formulario no pide teléfono): el usuario elige el chat.
- El asunto del correo es fijo, así que la línea «Asunto: …» que escribe el modelo queda en el cuerpo.
- Los textos largos generan enlaces `mailto:` de más de 2.000 caracteres, que algunos gestores de correo de Windows recortan; «Copiar» sigue disponible.
- Los correos (restablecimiento de contraseña) y la documentación de la API (`/docs`) aún no están traducidos.
- El formulario de contacto no entrega mensajes (modo `demo`).
- Las páginas legal y de privacidad muestran «Pendiente de completar» hasta que existan los datos del titular.
- El ADR 0017 (docs/adr/0017-visual-identity-and-interface-languages.md) sigue vigente para la identidad visual y las pantallas de acceso; su parte de idiomas de la interfaz (solo español) está marcada como sustituida por la sección 5.3 de este documento.

Documento propiedad de Kodexia — Software & ERP Solutions. Todos los derechos reservados.